Marketing Analytics

R Onboarding

Nils Wlömert
Vienna University of Economics and Business (WU)

October 2026

What you will have at the end

- R and RStudio running — or a **Posit Cloud** account
- A course folder set up as an **RStudio Project** with subfolders for code, data, and graphics
- A script that **loads the course data** without errors
- A short **report knitted to HTML** — the format of your assignments

Self-paced, about 2–3 hours, before session 2. These slides summarize the onboarding module in the course book — the book has all details, screenshots, and the complete R code.

Why R?

- **Free**, runs on every operating system
- A standard tool of data analysis in **research and industry**
- **Reproducible**: code can be re-run, checked, corrected, and shared — mouse clicks in a spreadsheet cannot

> **AI writes R code cheaply** — that does not make R skills obsolete, it shifts them: the scarce skill is no longer *typing* code, but **reading and checking** it.

Setting up

Two routes to a working setup

Route A (recommended): local installation

1. **R** from CRAN: https://cran.r-project.org
2. **RStudio Desktop**: https://posit.co/download/rstudio-desktop/

Install **in this order**, with the default settings. RStudio is built on top of R: R without RStudio works — RStudio without R does not.

Route B (fallback): Posit Cloud

RStudio in the browser — no installation: https://posit.cloud → *New Project* → *New RStudio Project*.

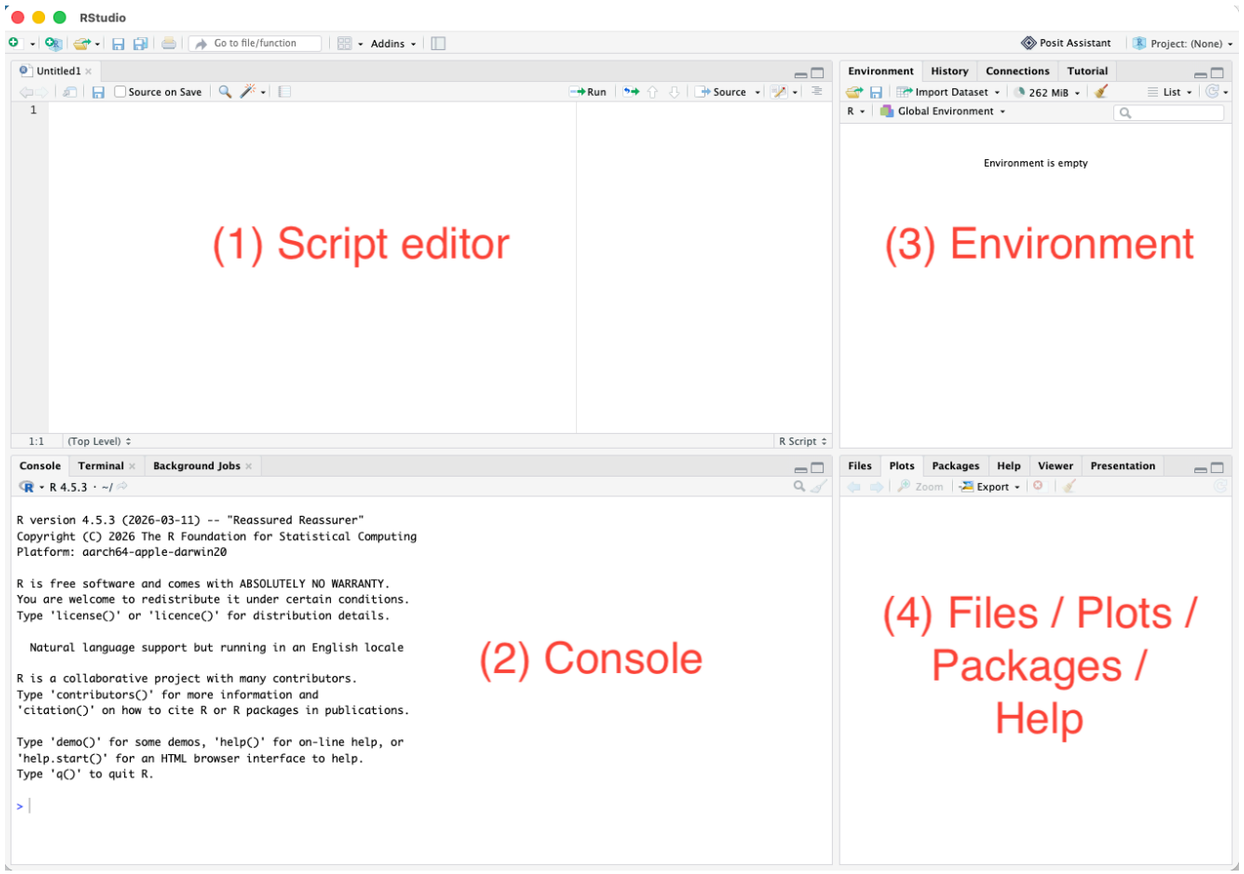
Useful if the installation fails or your computer is managed by an employer. Free accounts have usage limits.

Known pitfalls

- **Synced folders** (OneDrive, iCloud, Dropbox) cause strange errors → keep the course folder in a normal local folder
- **Special characters in paths** (umlauts, spaces) can break package installation on Windows → e.g., `C:/marketing-analytics`
- **macOS:** installation errors mentioning `xcrun` → run `xcode-select --install` in the Terminal app
- **Old R versions:** update — many packages need a recent R

> **macOS: "Operation not permitted"** when knitting or reading files → RStudio lacks permission to access the folder.
>
> *System Settings* → *Privacy & Security* → *Files & Folders* (or *Full Disk Access*): allow RStudio, then **restart RStudio** — or keep the course folder directly in your home folder (`~/marketing-analytics`).

RStudio in ten minutes



Pane	Purpose
Script editor	*Write* and keep your code (`.R` file)
Console	R *executes* code and prints results
Environment	Objects in your current session
Files / Plots / Help	Files, charts, help pages

> **Always write code in a script.** Run a line with **Ctrl + Enter** (Mac: **Cmd + Enter**).

Work in an RStudio Project

File → New Project → New Directory → New Project

- R automatically looks for files **in the project folder** — no more “file not found”
- Start every work session by **opening the project** (`.Rproj` file)
- No `setwd("C:/Users/…")`: hard-coded paths break on every other computer

Change one setting now: *Tools → Global Options → General*

- “Save workspace to .RData on exit” → **Never**
- Untick “Restore .RData into workspace at startup”

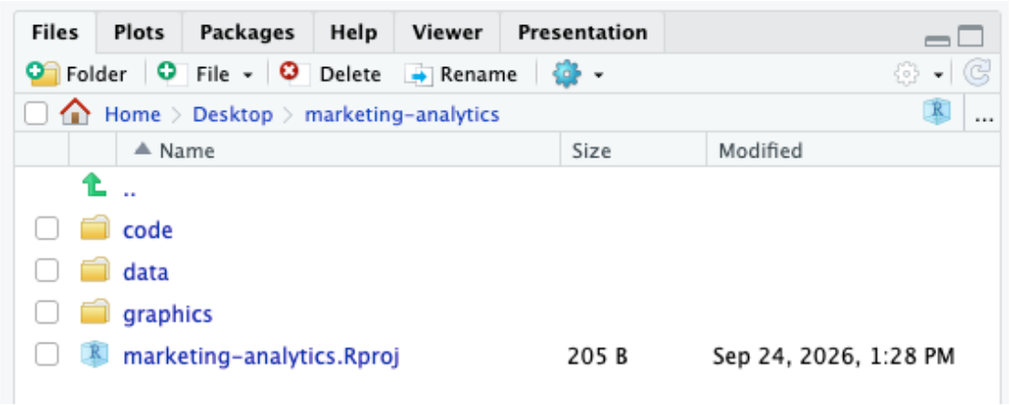
Your **script** is the record of your work — not the workspace.

A tidy folder structure

```
marketing-analytics/            <- project folder
├── marketing-analytics.Rproj
├── code/                       <- .R and .Rmd files
├── data/                       <- .csv, .xlsx, .sav
└── graphics/                   <- saved charts
```

Refer to files by their **relative path**:

- `read_csv2("data/music_data_fin.csv")`
- `ggsave("graphics/genre_shares.png")`



> **Never edit raw data by hand · no spaces or brackets in file names:** `assignment-1.Rmd`, not `Assignment 1 (final).Rmd`

First steps in R

How to run code

1. Create a script: *File* → *New File* → *R Script* → save as `code/first-steps.R`
2. **Type** (or copy) the code into the script — typing it yourself helps you remember
3. Cursor in the line → **Ctrl + Enter** (Mac: **Cmd + Enter**) or the **Run** button; select several lines to run them together; whole script: **Ctrl/Cmd + Shift + Enter**
4. Compare the result in the **console** with the output in the book

> **The console shows `+` instead of `>`?** R is waiting for you to finish a command — usually a missing closing bracket or quotation mark. Complete the command, or press **Esc** to abort.

Objects and assignment

```
streams <- 163608            # assign a number to the object 'streams'
streams * 2                  # objects can be used in calculations
```

```
[1] 327216
```

```
artist <- "Imagine Dragons"  # text in quotation marks
artist
```

```
[1] "Imagine Dragons"
```

> `<-` assigns (shortcut **Alt + -**, Mac: **Option + -**) · `#` starts a comment · R is **case-sensitive**: `streams` ≠ `Streams` · names: lower case with underscores, e.g., `mean_streams`

Functions and arguments

```r
seq(from = 1, to = 10, by = 3)   # arguments named explicitly
```

```
[1]  1  4  7 10
```

```r
seq(1, 10, 3)                    # same result, default order
```

```
[1]  1  4  7 10
```

```r
round(3.14159, digits = 2)
```

```
[1] 3.14
```

> Named arguments can come in any order · **help** for any function: `?seq` or **F1** on the function name · **Tab** shows completions while you type

Vectors and data types

```
1 top_streams <- c(163608, 126687, 120480)
2 top_genres  <- c("Dance", "Alternative", "Latino")
3 mean(top_streams)
```

```
[1] 136925
```

```
1 class(top_genres)
```

```
[1] "character"
```

Type	Example
numeric	`163608`, `3.14`
character	`"Dance"`
factor	`factor(c("Pop", "Rock"))`
logical	`TRUE`, `FALSE`
Date	`as.Date("2026-10-12")`

Convert with `as.numeric()`, `as.character()`, `as.factor()`, `as.Date()`

Data frames: R’s tables

```r
top_tracks <- data.frame(
  artist  = c("Ed Sheeran", "Imagine Dragons", "J. Balvin"),
  streams = c(163608, 126687, 120480)
)
top_tracks$streams     # $ extracts one column
```

```
[1] 163608 126687 120480
```

```r
str(top_tracks)        # structure: rows, columns, types
```

```
'data.frame':   3 obs. of  2 variables:
 $ artist : chr  "Ed Sheeran" "Imagine Dragons" "J. Balvin"
 $ streams: num  163608 126687 120480
```

Packages

```r
# once per computer:
install.packages(c("tidyverse", "psych",
                   "readxl", "haven", "rmarkdown"))
```

```r
# in every session, at the top of every script:
library(tidyverse)
```

install = once per computer · **library** = every session

`could not find function "…"`? → the package is probably not loaded. Without loading: `dplyr::filter()`

Three dialects of R

“Keep the pop tracks and compute their average number of streams”:

```r
pop <- music_data[music_data$genre == "Pop", ]          # base R
mean(pop$streams)

music_data |> filter(genre == "Pop") |>                 # tidyverse
  summarise(mean_streams = mean(streams))

as.data.table(music_data)[genre == "Pop", mean(streams)] # data.table
```

Dialect	Strength	Drawback
Base R	always available	hard to read
tidyverse	reads like a sentence; `ggplot2`	needs packages
data.table	very fast	cryptic for beginners

This course uses the tidyverse: readable pipelines are easy to *check* — whether you or an AI wrote them. Be able to *read* base R.

Keyboard shortcuts worth learning

Action	Windows	Mac
Run line / selection	Ctrl + Enter	Cmd + Enter
Run whole script	Ctrl + Shift + Enter	Cmd + Shift + Enter
Assignment <–	Alt + -	Option + -
Pipe |>	Ctrl + Shift + M	Cmd + Shift + M
Insert code chunk	Ctrl + Alt + I	Cmd + Option + I
Knit	Ctrl + Shift + K	Cmd + Shift + K
Comment lines	Ctrl + Shift + C	Cmd + Shift + C

Native pipe: *Tools → Global Options → Code → Editing* → tick **"Use native pipe operator, |>"** — otherwise the shortcut inserts %>%.

Own shortcuts: *Tools → Modify Keyboard Shortcuts…*

Data and reports

Loading the course data

```r
# directly from the course repository on GitHub (needs internet)
music_url <- "https://raw.githubusercontent.com/wu-rds/MA2026/main/data/music_data_fin.csv"
music_data <- read_csv2(music_url)

# or: download once into the project's data folder
music_data <- read_csv2("data/music_data_fin.csv")
```

```r
dim(music_data)   # number of rows and columns
```

```
[1] 66796    31
```

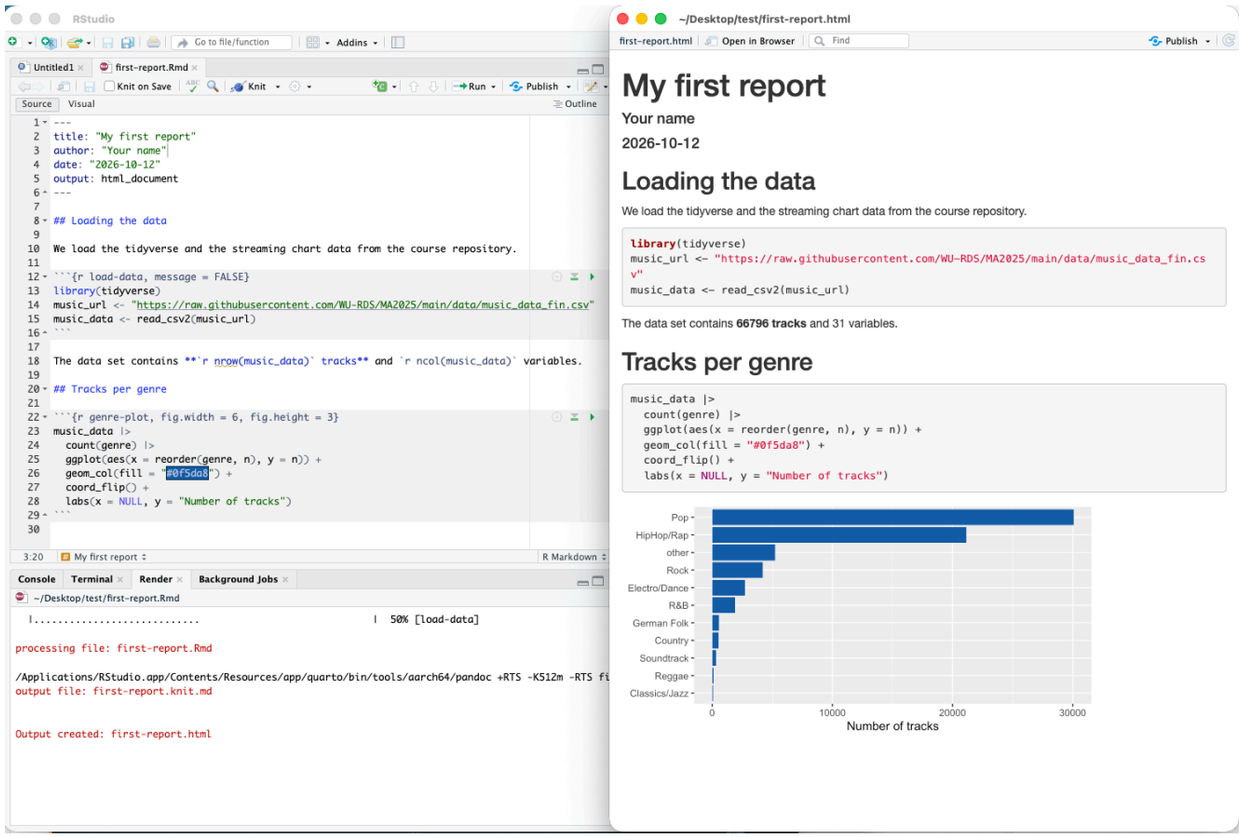
> Why `read_csv2()` and not `read_csv()`? → first lesson of Chapter 2.

Your first reproducible report

File → New File → R Markdown… → HTML → save as `code/first-report.Rmd` → **Knit**

Three ingredients:

- **Header** between `---`: title, author, output
- **Text** in Markdown: `**bold**`, `*italic*`, `# Heading`
- **Code chunks** between ```` ```{r} ```` and ```` ``` ````
- **Inline code**: `` `r nrow(music_data)` `` puts results into the text



Two rules that save hours

The golden rule: a report knits in a **fresh R session**. Everything it needs — `library()`, loading the data, every step — must be **in the report**. Works in the console but not when knitting? You rely on something that exists only in your session.

Keep the chunk delimiters intact: every chunk needs **three backticks** at the start (```` ```{r} ````) and at the end (```` ``` ````). One deleted backtick → the report no longer knits. Insert chunks with the shortcut.

File paths in reports: a report in `code/` finds the data at `"../data/music_data_fin.csv"` (`..` = one folder up).

Getting help — and working with AI

Reading error messages

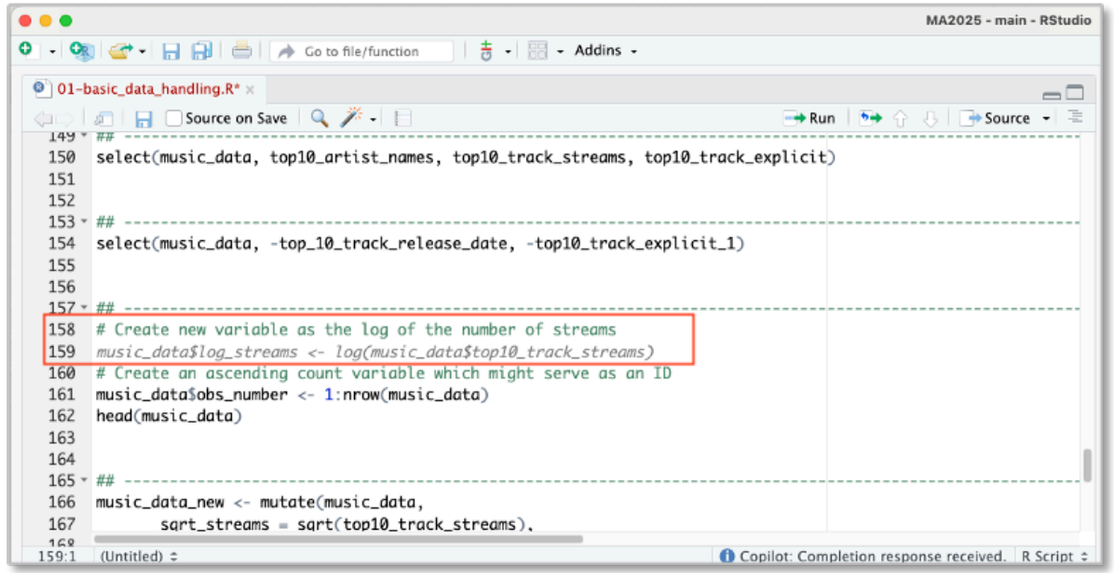
Message	What it usually means
`could not find function "…"`	Package not loaded — or function name misspelled
`object '…' not found`	Typo, wrong capitalization, or the creating line was not run
`cannot open file '…'`	Wrong place or wrong name — are you in your RStudio Project?
`unexpected symbol` / `unexpected ')'`	Missing comma, bracket, or quotation mark
`there is no package called '…'`	Not installed yet → `install.packages("…")`

> Errors are not a sign that you are doing something wrong — they are R's way of telling you **where to look**. Read the message first.

AI assistants: how they help

Chat assistants — ChatGPT, Claude, Gemini, Perplexity

Inside RStudio — GitHub Copilot suggests code as you type: *Tools → Global Options → Copilot* (GitHub account; free for verified students via GitHub Education)



Good at: standard code (wrangling, charts, loops) · explaining functions and errors · debugging · suggesting alternatives

Less reliable at: statistical reasoning and **interpreting results** — fluent even when wrong

Asking well

```
I am working in R with the following data frame:

> str(music_data)
'data.frame': 66796 obs. of 31 variables:
 $ isrc        : chr "BRRGE1603547" ...
 $ danceability: num 50.9 35.3 68.3 ...
 $ valence     : num 65.1 43.7 43.4 ...

Please provide the code for a publication-ready
scatterplot between valence and danceability with
pink hollow circles and a red regression line, and
label the line with the regression coefficient.
```

- **Give context:** `str()` output, code, the *complete* error message
- **Be specific** about what you want
- **Start small**, then add complexity
- **Ask for explanations**, not just fixes

Example: from prompt to chart

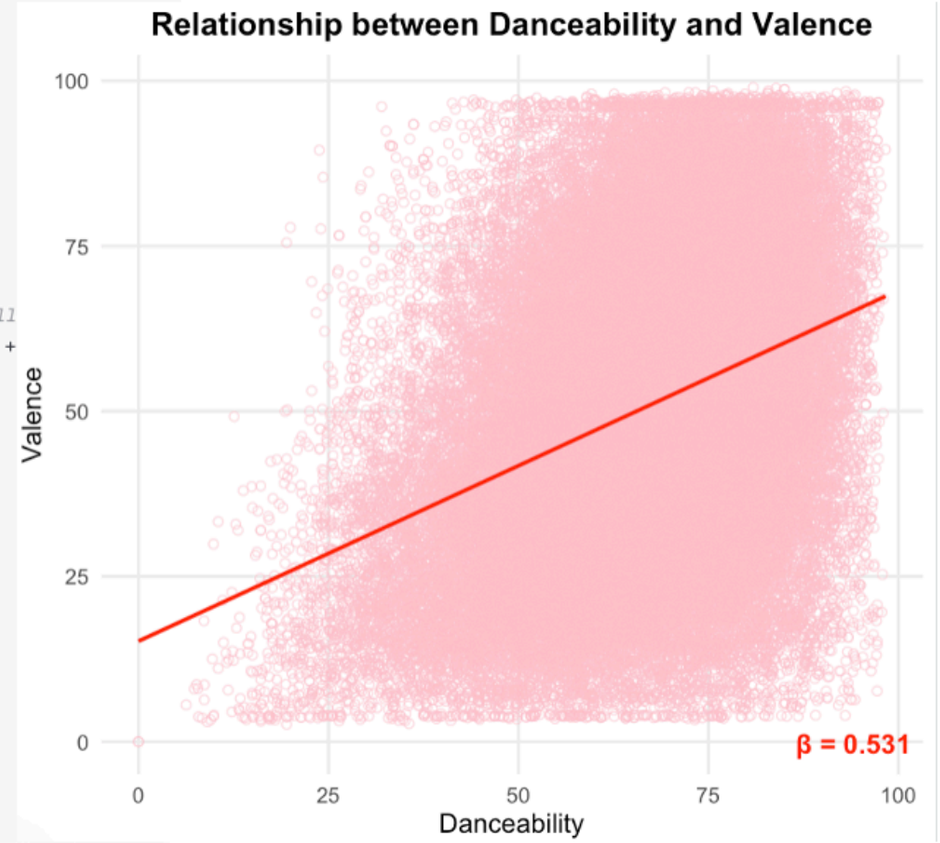
r

Copy code

```r
# Load necessary package
library(ggplot2)

# Fit linear model
model <- lm(valence ~ danceability, data = music_data)
coef_value <- coef(model)[2]

# Create scatterplot
ggplot(music_data, aes(x = danceability, y = valence)) +
  geom_point(color = "pink", shape = 1, size = 2, alpha = 0.6) +  # holl
  geom_smooth(method = "lm", se = FALSE, color = "red", linewidth = 1) +
  annotate("text",
           x = Inf, y = -Inf,
           label = paste0("β = ", round(coef_value, 3)),
           hjust = 1.1, vjust = -1.1,
           size = 5, color = "red", fontface = "bold") +
  labs(
    x = "Danceability",
    y = "Valence",
    title = "Relationship between Danceability and Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    panel.grid.minor = element_blank()
  )
```



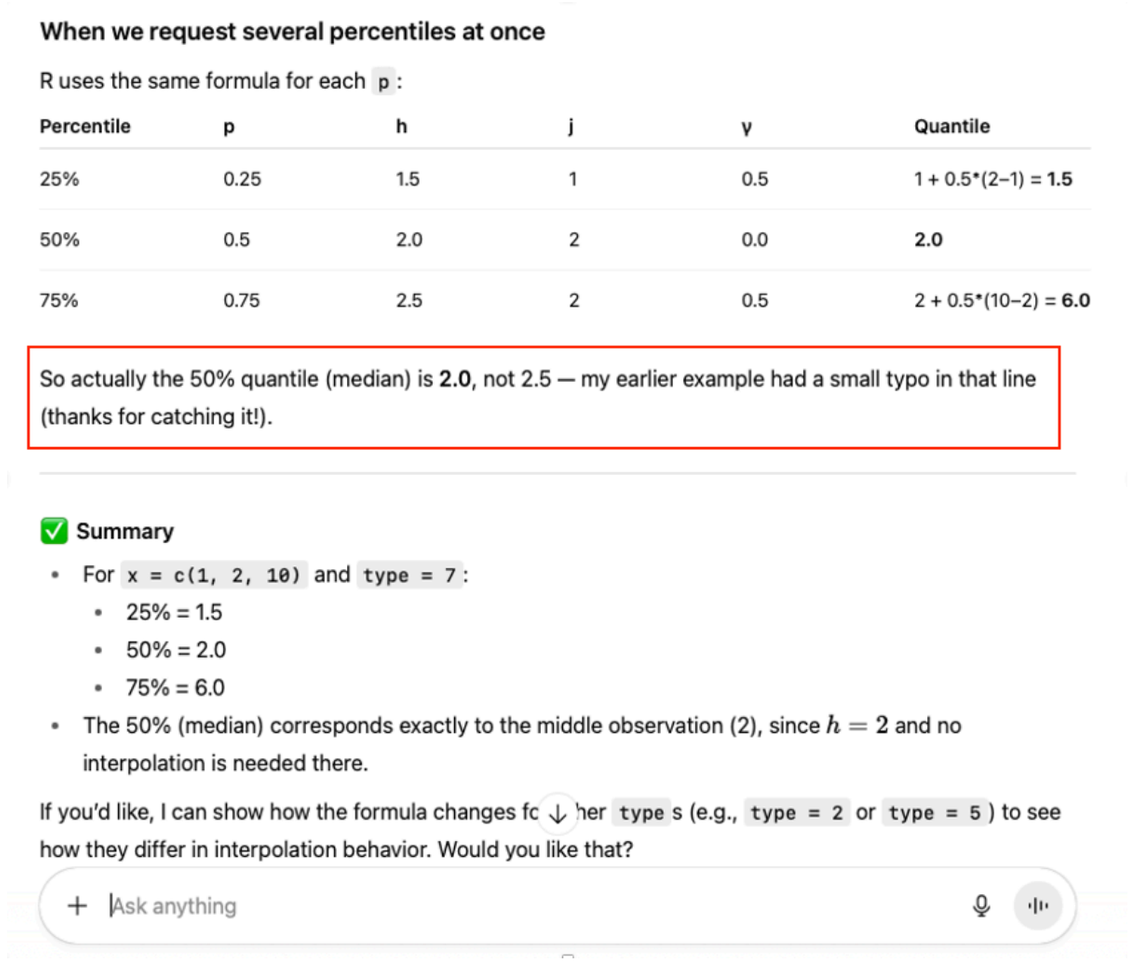
Debugging works too: `could not find function "geom_pont"` + your code → the assistant spots the typo (`geom_point()`) and returns the corrected code.

Always verify: assistants can be confidently wrong

An assistant explained R's percentile calculation correctly — and then reported the **median of 1, 2, 10 as 2.5**. Five seconds in R:

```r
x <- c(1, 2, 10)
quantile(x, c(0.25, 0.5, 0.75))
```

```
25% 50% 75%
1.5 2.0 6.0
```



When we request several percentiles at once

R uses the same formula for each `p`:

Percentile	p	h	j	γ	Quantile
25%	0.25	1.5	1	0.5	1 + 0.5*(2−1) = **1.5**
50%	0.5	2.0	2	0.0	**2.0**
75%	0.75	2.5	2	0.5	2 + 0.5*(10−2) = **6.0**

So actually the 50% quantile (median) is **2.0**, not 2.5 — my earlier example had a small typo in that line (thanks for catching it!).

✅ **Summary**

- For `x = c(1, 2, 10)` and `type = 7`:
  - 25% = 1.5
  - 50% = 2.0
  - 75% = 6.0
- The 50% (median) corresponds exactly to the middle observation (2), since $h = 2$ and no interpolation is needed there.

If you'd like, I can show how the formula changes fo her `type`s (e.g., `type = 2` or `type = 5`) to see how they differ in interpolation behavior. Would you like that?

Ask anything

What saved the day was not a better prompt, but **running the code and checking the result**.

Using AI assistants mindfully

- **Foundations first** — you need to know what you are asking, and recognize a wrong answer
- **Critical thinking** — check outputs for correctness, efficiency, and reproducibility
- **Accelerate, don’t bypass, learning** — if you could not explain the code to someone else, you are not done
- **Transparency** — disclose AI use in your assignments; be able to explain every line you submit

Checkpoint: ready for session 2?

1. Open your RStudio Project — subfolders `code`, `data`, `graphics`
2. Download the course data into `data/`
3. New R Markdown file: one chunk that loads the tidyverse and the data
4. One sentence with the number of tracks — via **inline code**
5. A second chunk with `str(music_data)`
6. **Knit** to HTML

> HTML opens and shows the structure of the data? **Your setup works.**
>
> Stuck? Forum on Canvas, your AI assistant — and the **setup clinic** at the start of session 2.